



所属专栏: [Linux](#)

作者主页: [某某](#)

Linux：线程概念于控制

概念

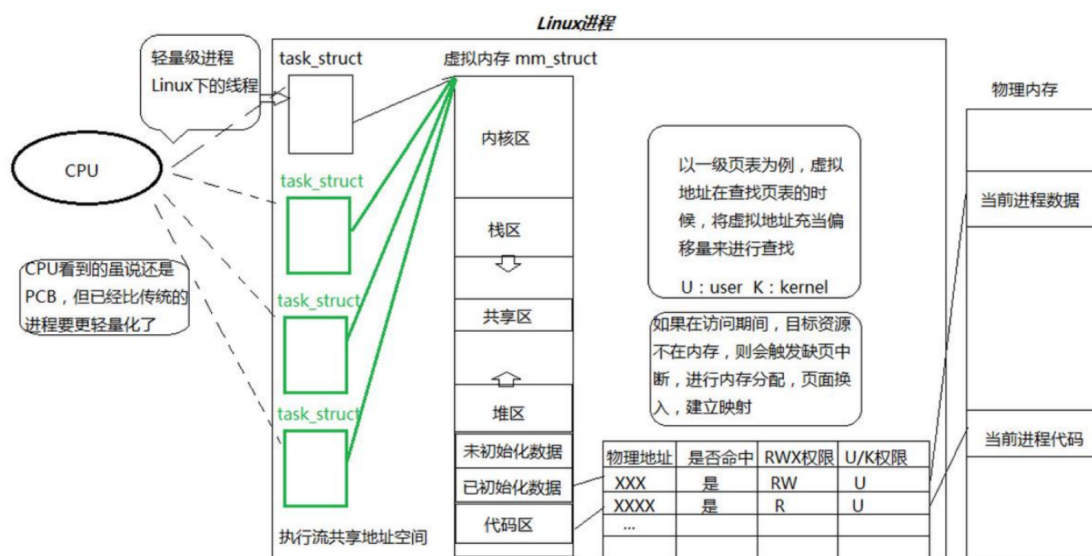
在一个程序里的一个执行路线就叫做线程 thread。更准确一点：线程是“一个进程内部的控制序列”

一切进程都至少有一个线程

线程在进程内部运行，本质是在进程地址空间运行

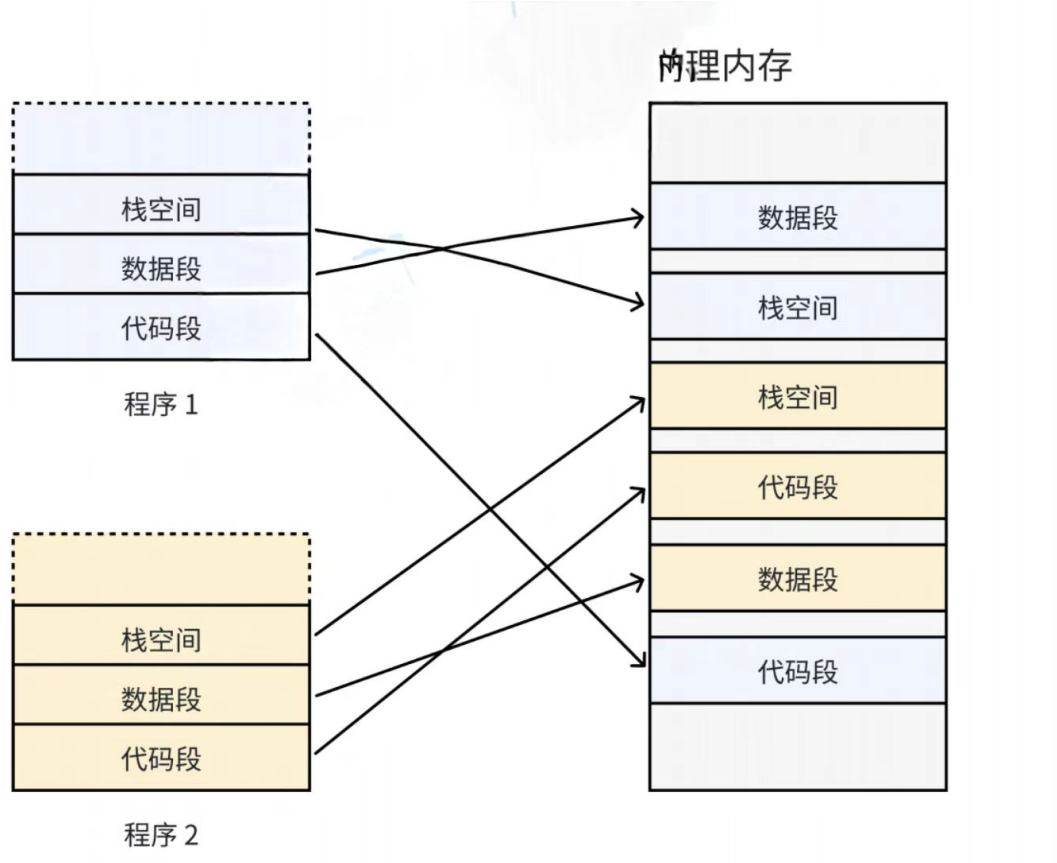
在 Linux 系统中，在 CPU 眼中，看到的PCB都要比传统的要轻量化

透过进程虚拟地址空间，可以看到进程的大部分资源，将进程资源合理分配给每个执行流，就形成了线程执行流。



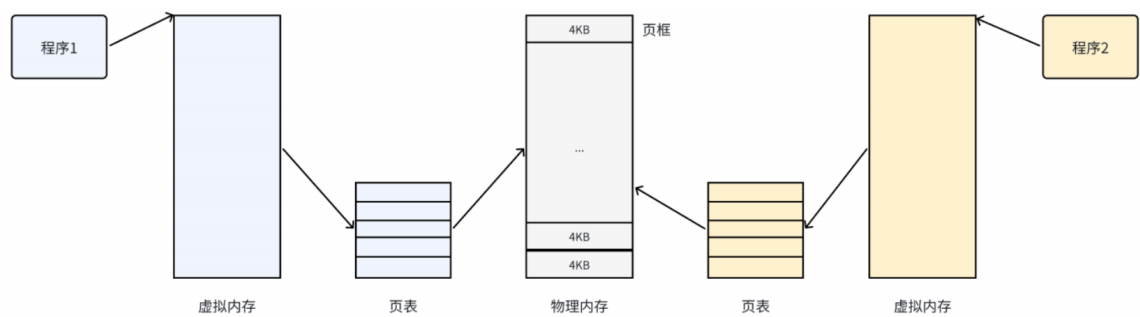
分页式储存管理

如果没有虚拟内存和分页机制，每一个用户在物理内存上的空间必须是连续的，如下图



因为每一个程序的代码、数据长度都是不一样的，按照这样的映射方式，物理内存会被分成各种离散的大小不同的块。有些程序会退出，它们占据的物理内存会被回收，一段时间过后，物理内存就变得非常碎片化，不易管理了。

所以我们希望操作系统给用户的空间是连续的，但是物理内存最好不要连续，那么虚拟内存和分页就出现了。



把物理内存按照一个固定的长度页框进行分割，叫做物理页。每一个页框包含一个物理页 `page`。页的大小就是页框的大小。32 位机器支持 4kb 的页，64 位机器支持 8kb 的页。

- 页框是一个储存区域
- 页是一个数据块，可以存放在任何页框中，或者磁盘中

有了这种机制，CPU 并非直接访问物理地址，而是通过虚拟地址空间来间接的访问物理内存地址。所谓的虚拟地址空间，是操作系统位每一个在执行的进程分配的一个逻辑地址，32 位机器上其范围为 0-4GB。

操作系统通过将虚拟地址空间与物理地址间建立映射关系：页表，这张表上记录了每一对页和页框的映射关系，能让 CPU 间接访问物理内存地址。

其思想就是**将虚拟内存下的逻辑地址空间分为若干的页，将物理内存空间分为若干页框，通过页表将连续的虚拟内存，映射到若干个不连续的物理内存页**。这样就解决了使用连续的物理内存造成的碎片问题。

物理内存管理

假设一个可用的物理内存有 4GB 的空间。按照一个页框的大小 4KB 划分，4GB 的空间就是 4GB/4KB = 1048576 个页框。有这么多的物理页，操作系统肯定是要对它们进行管理的，os 要知道哪些页被使用，哪些页在空闲。

Linux内核里面使用了 `struct page` 结构描述系统中的每个物理页，为了节省内存（这个结构体本身也是在内核，也是在内存里面的）这里面使用了大量的联合体 `union`

```
1  /* include/linux/mm_types.h */
2  struct page
3  {
4      /* 原子标志，有些情况下会异步更新 */
5      unsigned long flags;
6      union
7      {
8          struct
9          {
10             /* 换出页列表，例如由zone->lru_lock保护的active_list */
11             struct list_head lru;
12             /* 如果最低为0，则指向inode
13              * address_space，或为NULL
14              * 如果页映射为匿名内存，最低为置位
15              * 而且该指针指向anon_vma对象
16              */
17             struct address_space *mapping;
18             /* 在映射内的偏移量 */
19             pgoff_t index;
20             /*
21              * 由映射私有，不透明数据
22              * 如果设置了PagePrivate，通常用于buffer_heads
23              * 如果设置了PageSwapCache，则用于swp_entry_t
24              * 如果设置了PG_buddy，则用于表示伙伴系统中的阶
25              */
26             unsigned long private;
27         };
28         struct
29         { /* slab, slob and slub */
30             union
31             {
32                 struct list_head slab_list; /* uses lru */
33                 struct
34                 { /* Partial pages */
35                     struct page *next;
36                 };
37             };
38             #ifdef CONFIG_64BIT
39             int pages; /* Nr of pages left */
40             int pobjects; /* Approximate count */
41             #else
```

```

40         short int pages;
41         short int pobjects;
42     #endif
43     };
44 };
45 struct kmem_cache *slab_cache; /* not slob */
46 /* Double-word boundary */
47 void *freelist; /* first free object */
48 union
49 {
50     void *s_mem; /* slab: first object */
51     unsigned long counters; /* SLUB */
52     struct
53     { /* SLUB */
54         unsigned inuse : 16; /* 用于SLUB分配器：对象的数目 */
55         unsigned objects : 15;
56         unsigned frozen : 1;
57     };
58 };
59 };
60 ...
61 };
62 union
63 {
64     /* 内存管理子系统中映射的页表项计数，用于表示页是否已经映射，还用于限制逆向映射搜索 */
65     atomic_t _mapcount;
66     unsigned int page_type;
67     unsigned int active; /* SLAB */
68     int units; /* SLOB */
69 };
70 ...
71 #if defined(WANT_PAGE_VIRTUAL)
72     /* 内核虚拟地址（如果没有映射则为NULL，即高端内存） */
73     void *virtual;
74 #endif /* WANT_PAGE_VIRTUAL */
75 ...
76 }

```

重要参数

1. **flags**: 用来存放页的状态。包括是不是脏的，是不是被锁定在内存中等等。**flags** 的每一位单独表示一种状态，所以他至少可以同时表示出32种不同的状态。这些标志定义在 `<linux/page-flags.h>` 中。其中一些比特位非常重要，如 **PG_locked** 用于指定页是否锁定，**PG_uptodate** 用于表示页的数据已经从块设备读取并且没有出现错误。
2. **_mapcount**: 表示在页表中有多少项指向该页，也就是这一页被引用了多少次。当计数值变为-1时，就说明当前内核并没有引用这一页，于是在新的分配中就可以使用它。
3. **virtual**: 是页的虚拟地址。通常情况下，它就是页在虚拟内存中的地址。有些内存（即所谓的高端内存）并不永久地映射到内核地址空间上。在这种情况下，这个域的值为 **NULL**，需要的时候，必须动态地映射这些页。

要注意的是 **struct page** 与物理页相关，而并非与虚拟页相关。而系统中的每个物理页都要分配一个这样的结构体，让我们来算算对所有这些页都这么做，到底要消耗掉多少内存。

算 `struct page` 占40个字节的内存吧，假定系统的物理页为 4KB 大小，系统有 4GB 物理内存。那么系统中共有页面 1048576 个（1兆个），所以描述这么多页面的 `page` 结构体消耗的内存只不过40MB，相对系统 4GB 内存而言，仅是很小的一部分罢了。因此，要管理系统中这么多物理页面，这个代价并不算太大。

要知道的是，页的大小对于内存利用和系统开销来说非常重要，页太大，页必然会剩余较大不能利用的空间（页内碎片）。页太小，虽然可以减小页内碎片的大小，但是页太多，会使得页表太长而占用内存，同时系统频繁地进行页转化，加重系统开销。因此，页的大小应该适中，通常为 512B - 8KB，windows 系统的页框大小为 4KB。

页表

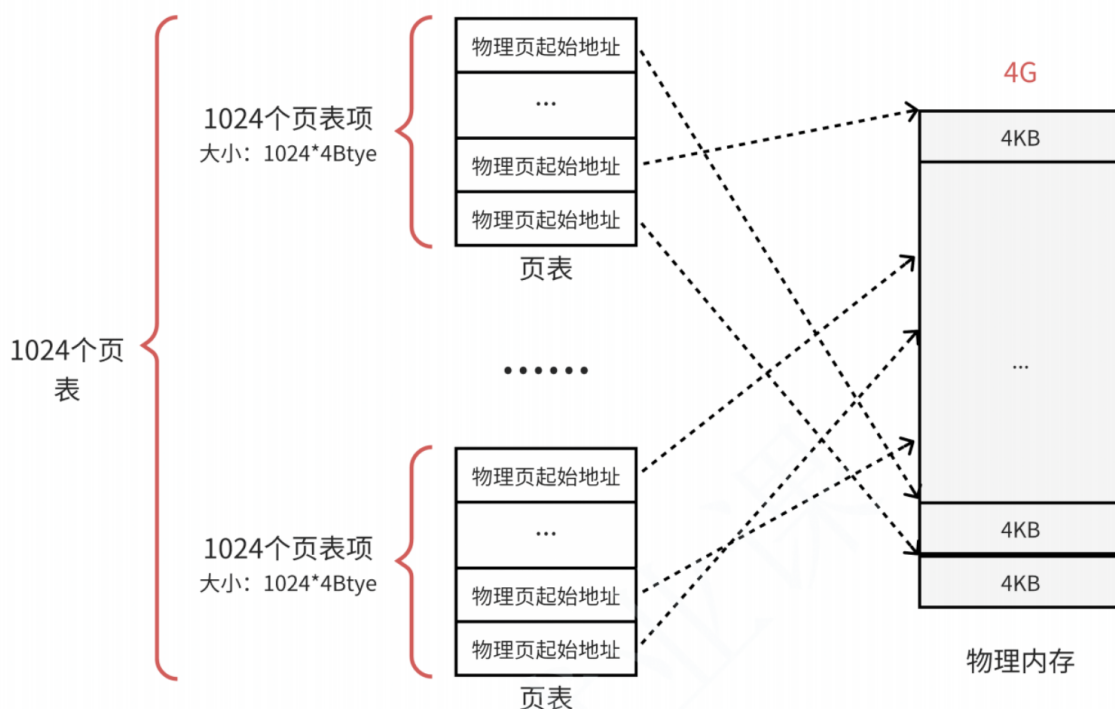
页表中的每一个表项都指向一个物理页的起始地址。在 32 位机器上要将 4GB 的空间全部用页表指向的话需要 $4GB/4KB = 1048576$ 个表项。

页表中的物理地址，和真实的物理页之间是随机映射关系，哪里可用就指向哪里。**最终使用的物理内存是离散的，但是虚拟地址是连续的。**处理器在访问数据、获取指令都是用的虚拟地址，最终都能通过页表找到物理地址。

我们可以算一下，在32位机器上，地址长度为4字节，页表占用的空间为 $1048576 * 4 = 4MB$ ，映射页表本身就要消耗 $4MB / 4KB = 1024$ 个物理页。每次新起一个进程，都要将这占用了1024个物理页的页表加载映射到内存，但是根据局部性原理，一个进程在一段时间内只需要访问某几个页就可以正常运行了。所以根本没有必要将所有的物理页都常驻内存。

解决这一问题的方法就是对页表再次分页，即多级页表。

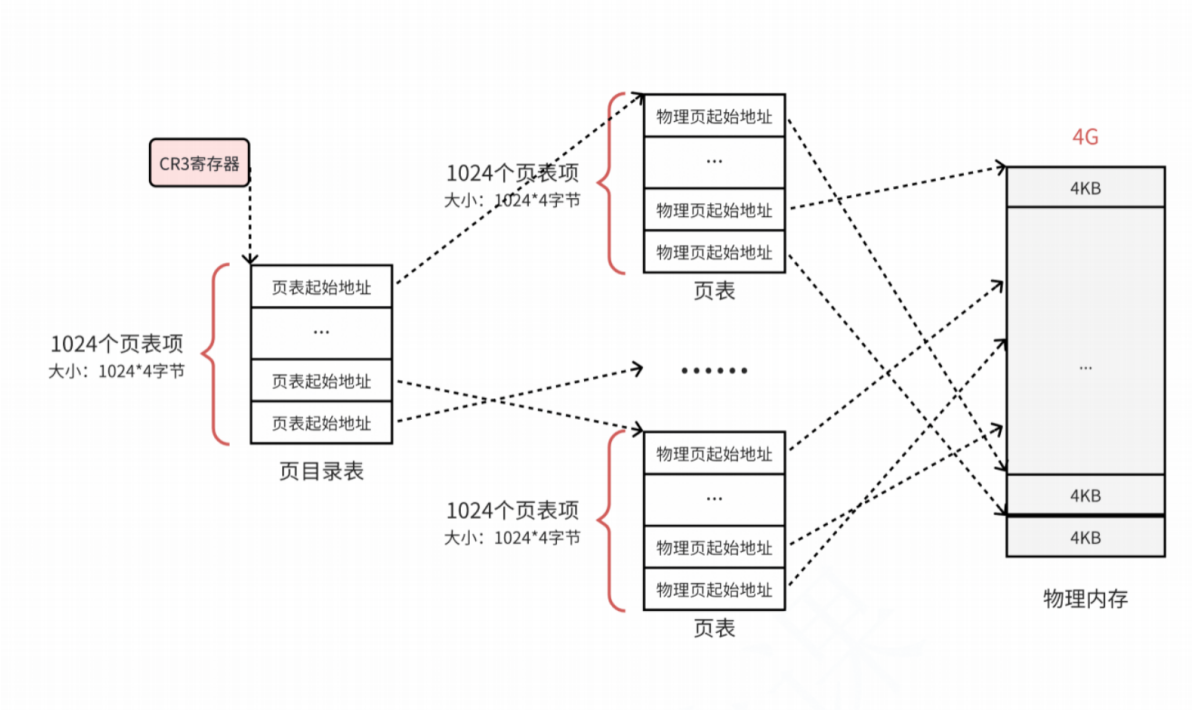
我们将单一的一个页表再次拆分为1024个更小的映射表。1024（每个表中的表项数）* 1024（表的个数）仍然可以覆盖4GB的物理内存空间。



这里的每一个页表就是真正的页表了，一共有1024个页表，一个页表占用4KB，一共也就是4MB。虽然和之前没区别，但是一个应用程序不可能把这些页表全部用完的，这样设计方便用多少，开多少。例如：一个程序的代码段、数据段、栈、一共只需要10MB的空间，那么使用3个页表就够用了。（一个页表能覆盖4MB的物理空间）

页目录结构

那么每一个页框都有一个页表项来指向，这1024个页表也需要被管理起来。管理页表的表称为页目录表，形成了二级页表：

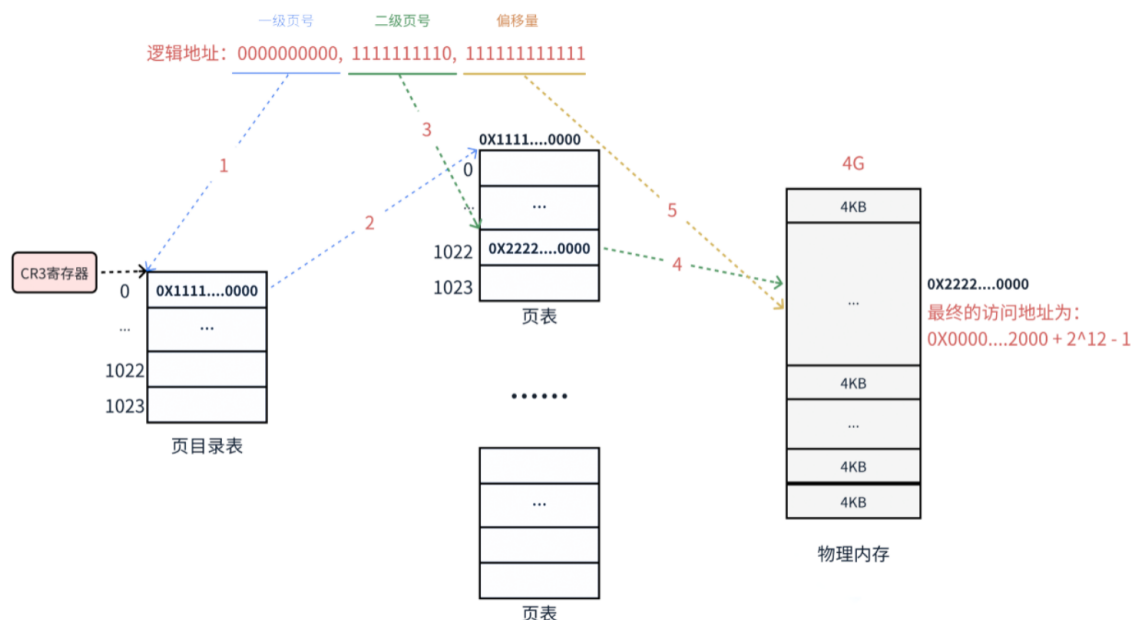


- 所有页表的物理地址都被页目录表指向
- 页目录的物理地址被 CR3 寄存器指向，此寄存器保存了当前正在执行任务的页目录地址。
- 所以在程序被加载的时候，不仅要为程序的内容分配物理内存，还需要为保存程序物理地址和虚拟地址映射的页目录和页表分配物理内存。

两级页表的地址转换

下面以一个逻辑地址为例。将逻辑地址 00000000000, 00000000001, 111111111111 转化为物理地址的过程：

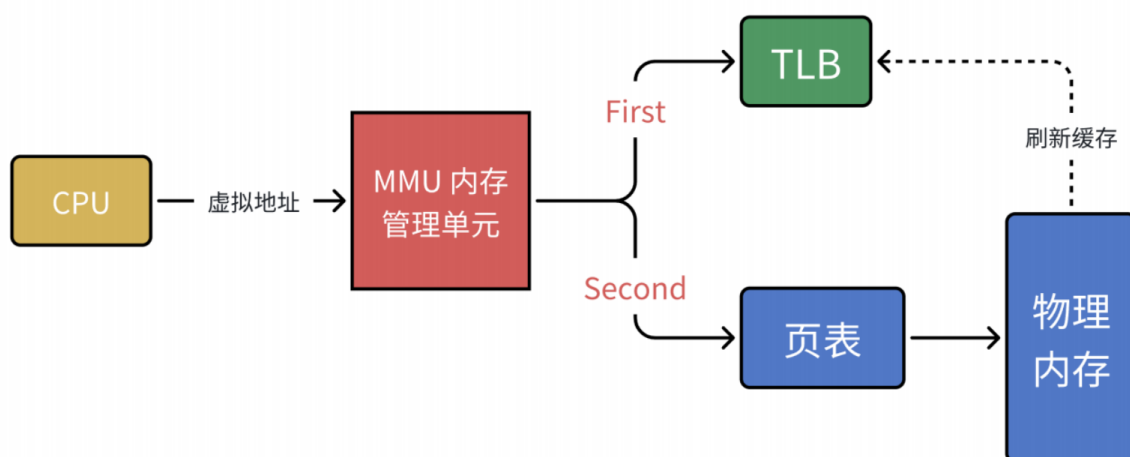
- 在32为处理器中，采用4KB页大小，则虚拟地址中的低12为为页偏移，剩下的高20位给页表分成两级，每个级占10位。
- CR3 寄存器读取页目录起始地址，再根据一级页号查页目录表，找到下一级页表在物理内存中存放位置。
- 根据二级页号查表，找到最终想要访问的内存块号。
- 结合页内偏移量得到物理地址。



- 一个物理页的地址，一定是对齐4KB的（最后的12位全为0），所以其实只需要记录物理页地址的高20位即可。
- 以上就是MMU的工作流程，Memory Manage Unit是一种硬件电路，其速度很快，主要工作是做内存管理，地址转换只是其工作之一。

那么这个工作流就没有缺点了吗？当然是有的！MMU进行两次查询确定物理地址，在确认了权限问题后，将这个物理地址发到总线，内存收到后开始读取数据并返回。当页表变为N级时需要经过N次检索+1次读写，查询效率就会变低。

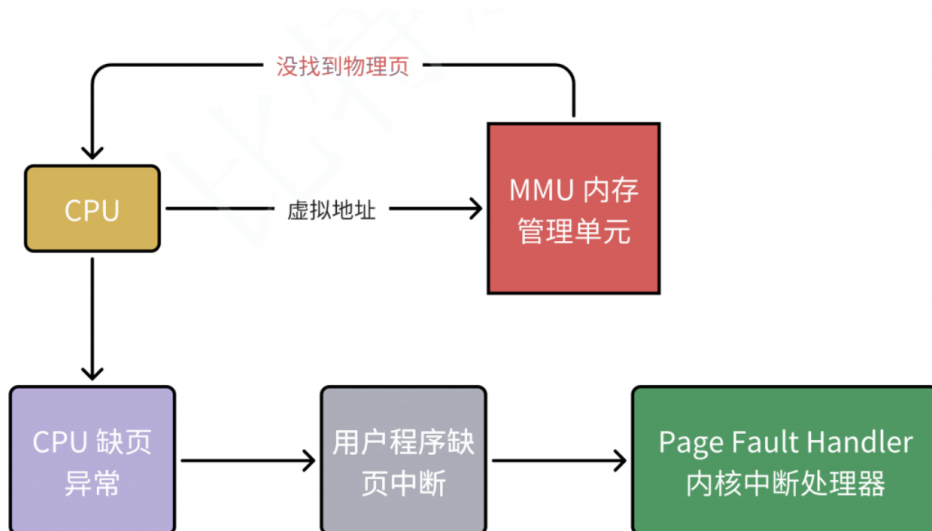
那么有没有提升效率的方法呢？有的，那就是添加一层中间层也就是TLB 江湖人称“快表”，其实也就是缓存。当CPU给MMU传新虚拟地址之后，MMU先去问TLB那边有没有，如果有就直接拿到物理地址发到总线给内存，齐活。但TLB容量比较小，难免发生Cache Miss，这时候MMU还有保底的老武器页表，在页表中找到之后MMU除了把地址发到总线传给内存，还把这条映射关系给到TLB，让它记录一下刷新缓存。



缺页异常

设想，CPU给MMU的虚拟地址，在TLB和页表都没有找到对应的物理页，该怎么办呢？其实这就是缺页异常Page Fault，它是一个由硬件中断触发的可以由软件逻辑纠正的错误。假如目标内存页在物理内存中没有对应的物理页或者存在但无对应权限，CPU就无法获取数据，这种情况下CPU就会报告一个缺页错误。由于CPU没有数据就无法进行计算，CPU罢工了用户进程也就出现了缺页中断，进程会

从用户态切换到内核态，并将缺页中断交给内核的 `Page Fault Handler` 处理。



缺页中断会交给 `PageFaultHandler` 处理，其根据缺页中断的不同类型进行不同的处理：

- `Hard Page Fault` 也被称为 `Major Page Fault`，翻译为硬缺页错误/主要缺页错误，这时物理内存中没有对应的物理页，需要 CPU 打开磁盘设备读取到物理内存中，再让 MMU 建立虚拟地址和物理地址的映射。
- `Soft Page Fault` 也被称为 `Minor Page Fault`，翻译为软缺页错误/次要缺页错误，这时物理内存中是存在对应物理页的，只不过可能是其他进程调入的，发出缺页异常的进程不知道而已，此时 MMU 只需要建立映射即可，无需从磁盘读取写入内存，一般出现在多进程共享内存区域。
- `Invalid Page Fault` 翻译为无效缺页错误，比如进程访问的内存地址越界访问，又比如对空指针解引用内核就会报 `segment fault` 错误中断进程直接挂掉。

Linux进程VS线程

线程优点

- 创建一个新线程的代价要比创建一个新进程小得多
- 与进程之间的切换相比，线程之间的切换需要操作系统做的工作要少很多
- 最主要的区别是线程的切换虚拟内存空间依然是相同的，但是进程切换是不同的。这两种上下文切换的处理都是通过操作系统内核来完成的。内核的这种切换过程伴随的最显著的性能损耗是将寄存器中的内容切换出。
- 另外一个隐藏的损耗是上下文的切换会扰乱处理器的缓存机制。简单的说，一旦去切换上下文，处理器中所有已经缓存的内存地址一瞬间都作废了。还有一个显著的区别是当你改变虚拟内存空间的时候，处理的页表缓冲 `TLB`（快表）会被全部刷新，这将导致内存的访问在一段时间内相当的低效。但是在线程的切换中，不会出现这个问题，当然还有硬件 `cache`。
- 线程占用的资源要比进程少很多
- 能充分利用多处理器的可并行数量
- 在等待慢速 `I/O` 操作结束的同时，程序可执行其他的计算任务
- 计算密集型应用，为了能在多处理器系统上运行，将计算分解到多个线程中实现
- `I/O` 密集型应用，为了提高性能，将 `I/O` 操作重叠。线程可以同时等待不同的 `I/O` 操作。

线程缺点

- 性能损失

一个很少被外部事件阻塞的计算密集型线程往往无法与其它线程共享同一个处理器。如果计算密集型线程的数量比可用的处理器多，那么可能会有较大的性能损失，这里的性能损失指的是增加了额外的同步和调度开销，而可用的资源不变。

- 健壮性降低

编写多线程需要更全面更深入的考虑，在一个多线程程序里，因时间分配上的细微偏差或者因共享了不该共享的变量而造成不良影响的可能性是很大的，换句话说线程之间是缺乏保护的。

- 缺乏访问控制

进程是访问控制的基本粒度，在一个线程中调用某些 `os` 函数会对整个进程造成影响。

- 编程难度提高

编写与调试一个多线程程序比单线程程序困难得多

线程异常

- 单个线程如果出现除零，野指针问题导致线程崩溃，进程也会随着崩溃
- 线程是进程的执行分支，线程出异常，就类似进程出异常，进而触发信号机制，终止进程，进程终止，该进程内的所有线程也就随即退出

用途

- 合理的使用多线程，能提高 CPU 密集型程序的执行效率
- 合理的使用多线程，能提高IO密集型程序的用户体验（如生活中我们一边写代码一边下载开发工具，就是多线程运行的一种表现）

进程和线程

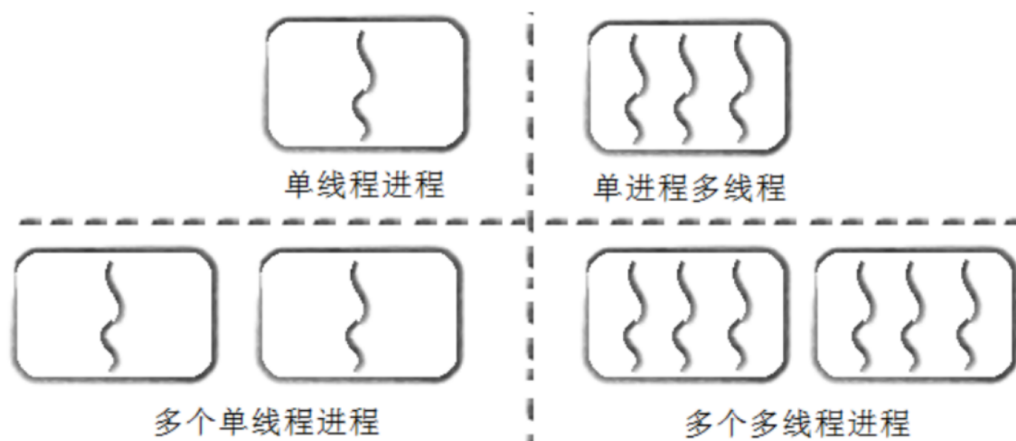
- 进程是资源分配的基本单位
- 线程是调度的基本单位
- 线程共享进程数据，但也拥有自己的一部分数据: 1、线程 ID 2、一组寄存器 3、栈 4、`errno` 5、信号屏蔽字 6、调度优先级

进程的多个线程共享

同一地址空间，因此 `Text Segment`、`Data Segment` 都是共享的，如果定义一个函数,在各线程中都可以调用，如果定义一个全局变量，在各线程中都可以访问到，除此之外，各线程还共享以下进程资源和环境：

- 文件描述符表
- 每种信号的处理方式(`SIG_IGN`、`SIG_DFL` 或者自定义的信号处理函数)
- 当前工作目录
- 用户 `id` 和组 `id`

进程和线程的关系如下图:



对于之前学习的进程和这个线程是相悖的吗？并不是，进程就可以看作一个有一个线程执行流的进程。

Linux线程控制

POSIX 线程库

- 与线程有关的函数构成了一个完整的系列，绝大多数函数的名字都是以“`pthread_`”打头的
- 要使用这些函数库，要通过引入头文件 `<pthread.h>`
- 链接这些线程函数库时要使用编译器命令的 `-lpthread` 选项

创建线程

```
1 功能：创建一个新的线程
2 原型：
3  int pthread_create(pthread_t *thread, const pthread_attr_t *attr, void *
  (*start_routine)(void*), void *arg);
4 参数：
5  thread: 返回线程ID
6  attr: 设置线程的属性，attr为NULL表示使用默认属性
7  start_routine: 是个函数地址，线程启动后要执行的函数
8  arg: 传给线程启动函数的参数
9  返回值：成功返回0；失败返回错误码
```

错误检查：

- 传统的一些函数是，成功返回0，失败返回-1，并且对全局变量 `errno` 赋值以指示错误
- `pthreads` 函数出错时不会设置全局变量 `errno`（而大部分其他 `POSIX` 函数会这样做）。而是将错误代码通过返回值返回
- `pthreads` 同样也提供了线程内的 `errno` 变量，以支持其它使用 `errno` 的代码。对于 `pthreads` 函数的错误，建议通过返回值来判定，因为读取返回值要比读取线程内的 `errno` 变量的开销更小

```
1  #include <unistd.h>
2  #include <stdlib.h>
3  #include <stdio.h>
4  #include <string.h>
5  #include <pthread.h>
```

```

6 void *rout(void *arg) {
7     int i;
8     for( ; ; ) {
9         printf("I'am thread 1\n");
10        sleep(1);
11    }
12 }
13 int main( void )
14 {
15     pthread_t tid;
16     int ret;
17     if ( (ret=pthread_create(&tid, NULL, rout, NULL)) != 0 ) {
18         fprintf(stderr, "pthread_create : %s\n", strerror(ret));
19         exit(EXIT_FAILURE);
20     }
21     int i;
22     for( ; ; ) {
23         printf("I'am main thread\n");
24         sleep(1);
25     }
26 }

```

```

1 #include <pthread.h>
2 // 获取线程ID
3 pthread_t pthread_self(void);

```

这个函数返回 `pthread_t` 类型的值，指代的是调用 `pthread_self` 函数的线程 `id`。这个 `id` 是 `pthread` 库给每个线程定义的进程内唯一标识，是 `pthread` 库维持的。没错，在内核层面根本没有“线程”这个概念，有的只是进程，只是 PCB 其它的东西都是库给我们封装的。Linux下是根据进程PCB重复利用，搞成了线程，windows 下则是另起炉灶，搞出来了一套新的东西。

由于每个进程都有自己独立的内存空间，故这个 `id` 的作用域是进程级的而不是系统级（内核不认识）。

其实 `pthread` 库中也是通过内核提供的系统调用 `clone...` 来创建线程的，而内核会为每个线程创建系统全局唯一的 `id` 来唯一标识这个线程。

我们可以使用 `PS` 命令查看线程信息：

```

1 $ ps -aL | head -1 && ps -aL | grep mythread
2      PID      LWP      TTY      TIME      CMD
3 2711838 2711838 pts/235 00:00:00 mythread
4 2711838 2711839 pts/235 00:00:00 mythread
5
6 -L 选项：打印线程信息

```

其中 `LWP` 就是真正的线程 `id` 可以使用 `gettid()` 调用，之前使用 `pthread_self` 得到的这个数实际上是一个地址，在虚拟地址空间上的一个地址，通过这个地址，可以找到关于这个线程的基本信息，包括线程ID，线程栈，寄存器等属性。

在 `ps -aL` 得到的线程 `ID`，有一个线程 `ID` 和进程 `ID` 相同，这个线程就是主线程，主线程的栈在虚拟地址空间的栈上，而其他线程的栈是在共享区（堆栈之间），因为 `pthread` 系列函数都是 `pthread` 库提供给我们的。而 `pthread` 库是在共享区的。所以除了主线程之外的其他线程的栈都在共享区。

线程终止

如果需要终止某个线程而不终止整个进程，可以有三种方法：

1. 从线程函数 `return`。这种方法对主线程不适用，从 `main` 函数 `return` 相当于调用 `exit`。
2. 线程可以调用 `pthread_exit` 终止自己。
3. 一个线程可以调用 `pthread_cancel` 终止同一进程中的另一个线程

`pthread_exit` 函数

```
1 功能：线程终止
2 原型：
3     void pthread_exit(void *value_ptr);
4 参数：
5     value_ptr: value_ptr 不要指向一个局部变量。
6 返回值：
7     无返回值，跟进程一样，线程结束的时候无法返回到它的调用者（自身）
```

注意：`pthread_exit` 函数中的输出型参数指向的内存必须是全局的，或者用 `malloc` 在堆上开辟的空间，不能指向在线程函数的栈上分配的空间，因为当其他线程得到这个返回值时，这个线程已经退出，会导致非法访问等内存问题。

`pthread_cancel` 函数

```
1 功能：
2     取消一个执行中的线程
3 原型：
4     int pthread_cancel(pthread_t thread);
5 参数：
6     thread: 线程ID
7 返回值：
8     成功返回0；失败返回错误码
```

线程等待

已经退出的线程，其地址空间没有被释放，仍然在进程地址空间内占据位置，新起来的线程也需要空间，所以需要回收退出线程的空间。

```
1 功能：
2     等待线程结束
3 原型：
4     int pthread_join(pthread_t thread, void **value_ptr);
5 参数：
6     thread: 线程ID
7     value_ptr: 它指向一个指针，后者指向线程的返回值
8 返回值：
9     成功返回0；失败返回错误码
```

调用该函数的线程将挂起等待，直到要 `join` 的线程终止。被 `join` 的线程以不同的方法终止，通过 `pthread_join` 得到的终止状态时不同的，总结如下：

1. 如果 `thread` 线程通过 `return` 返回，`value_ptr` 所指向的单元里存放的是 `thread` 线程函数的返回值。

- 2. 如果 `thread` 线程被别的线程调用 `pthread_cancel` 异常终掉, `value_ptr` 所指向的单元里存放的是常数 `PTHREAD_CANCELED`。
- 3. 如果 `thread` 线程是自己调用 `pthread_exit` 终止的, `value_ptr` 所指向的单元存放的是传给 `pthread_exit` 的参数。
- 4. 如果对 `thread` 线程的终止状态不感兴趣, 可以传 `NULL` 给 `value_ptr` 参数。

分离线程

默认情况下, 新创建的线程是 `joinable` 的, 线程退出后, 需要对其进行 `pthread_join` 操作, 否则无法释放资源, 从而造成系统泄漏。

如果不关心线程的返回值, `join` 是一种负担, 这个时候, 我们可以告诉系统, 当线程退出时, 自动释放线程资源。

```
1 | int pthread_detach(pthread_t thread);
```

可以是线程组内对其它线程进行分离, 亦可以是线程自己分离

```
1 | pthread_detach(pthread_self());
```

线程被分离了就不是 `joinable` 的了

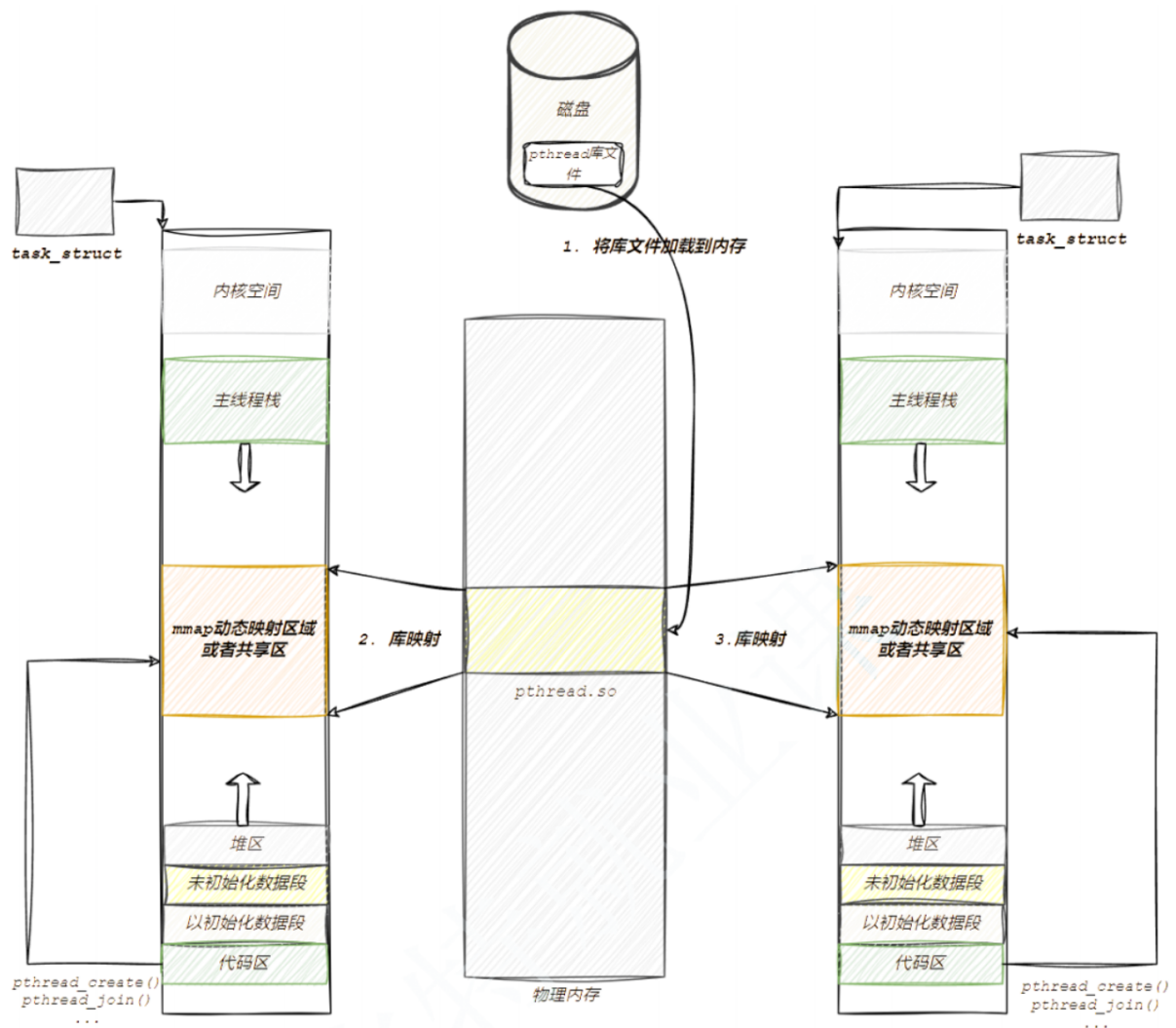
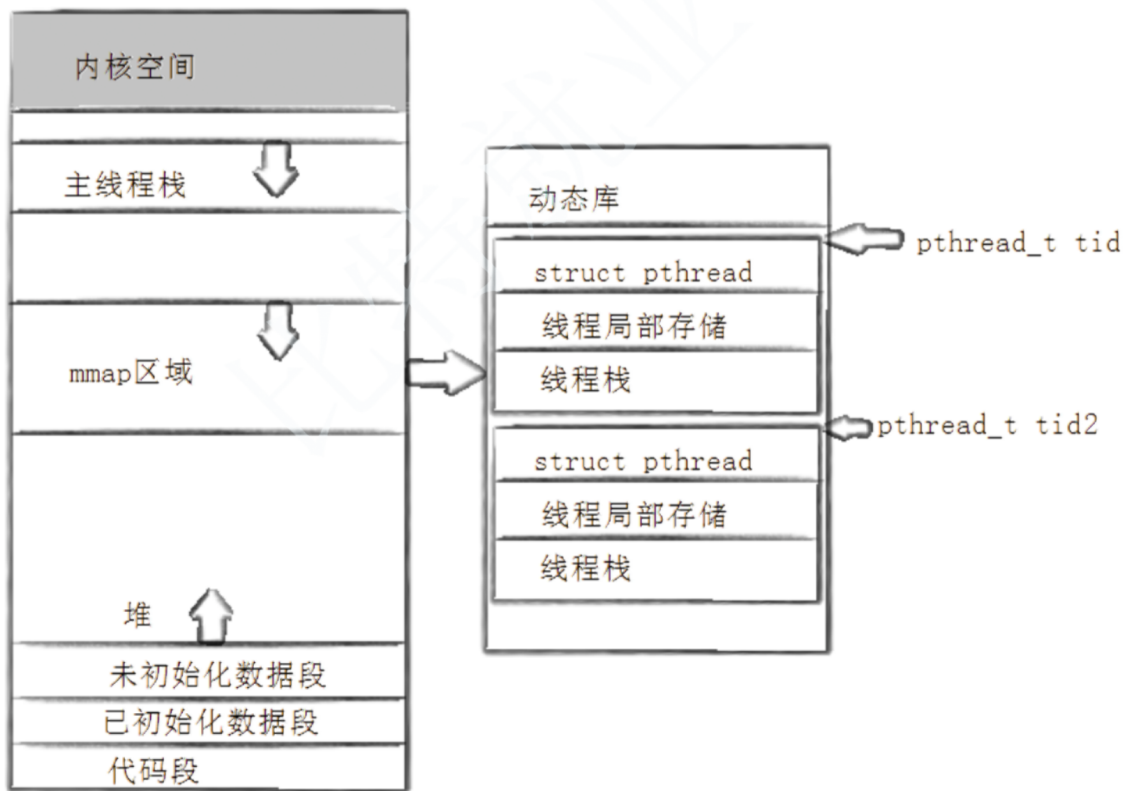
线程ID及进程地址空间布局

- `pthread_create` 函数会产生一个线程ID, 存放在第一个参数指向的地址中。该线程ID和前面说的线程ID不是一回事。前面讲的线程ID (LWP) 属于进程调度的范畴。因为线程是轻量级进程, 是操作系统调度器的最小单位, 所以需要数值来唯一表示该线程。
- `pthread_create` 函数第一个参数指向一个虚拟内存单元, 该内存单元的地址即为新创建线程的线程ID, 属于 `NPTL` 线程库的范畴。
- 线程库的后续操作, 就是根据该线程ID来操作线程的。线程库 `NPTL` 提供了 `pthread_self` 函数, 可以获得线程自身的ID:

```
1 | pthread_t pthread_self(void);
```

特征	LWP	<code>pthread_self()</code> 返回值
作用范围	内核空间 (系统级调度)	用户空间 (线程库级操作)
唯一性	全局唯一 (整个系统)	进程内唯一
数据类型	<code>pid_t</code> (通常为整数)	<code>pthread_t</code> (具体实现依赖系统)
获取方式	<code>syscall(SYS_gettid)</code> 或 <code>gettid()</code>	<code>pthread_self()</code>
用途	内核调度、系统调用、信号处理	线程库函数参数 (如 <code>pthread_equal</code>)

关于 `pthread_t` 的类型, 取决于具体实现, 对于 `Linux` 目前实现的 `NPTL` 而言, `pthread_t` 类型的线程ID, 本质就是一个进程地址空间上的一个地址。



线程封装

最新线程封装具体参考：

[25/Udp/chat/thread.hpp · 钦某/Code - 码云 - 开源中国 \(gitee.com\)](#)

[Code/25/Udp/chat/thread.hpp at master · QinMou000/Code \(github.com\)](#)

没什么好说的，注意 detach 的顺序就行，造一次轮子就行了。